

**МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА
И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ
Государственное бюджетное профессиональное
образовательное учреждение Нижегородской области
«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»**

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

**Анализ архитектуры и требований к взаимодействию микросервисов
интернет-магазина.**

Разработчик:

Студентка группы 24-ИС

Бакуменко Виктория Игоревна

Преподаватель:

Торопов Александр Вениаминович

2026 г.

Требования к компонентам и их взаимодействия

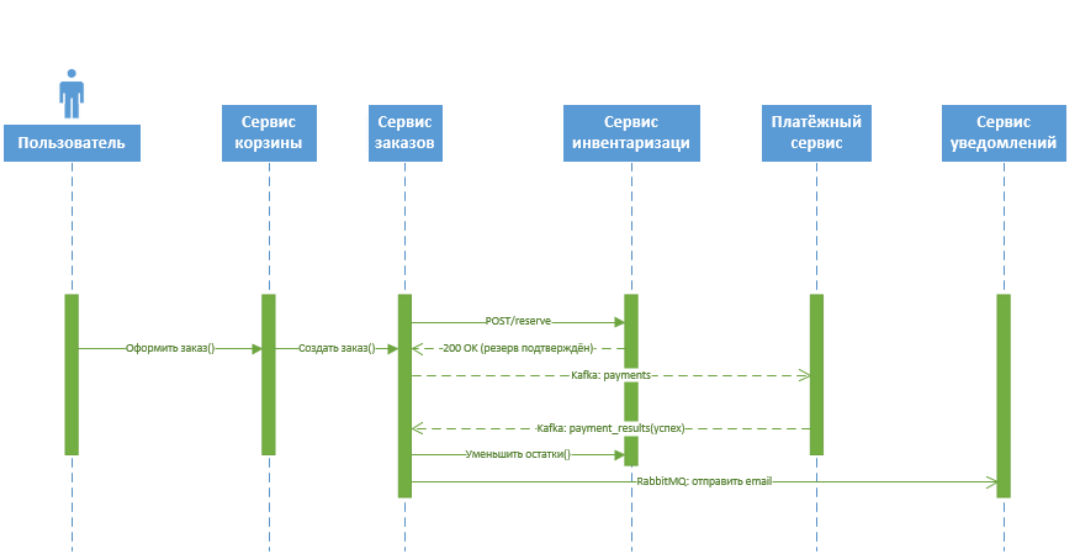
В ходе анализа ТЗ микросервисной архитектуры интернет-магазина были выделены следующие компоненты:

- 1. Сервис корзины (Cart Service):** отвечает за управление временными данными о выбранных пользователем товарах. Хранит данные в быстрослужащем кэш-хранилище Redis.
- 2. Сервис заказов (Order Service):** центральный оркестратор бизнес-процесса покупки. Создает заказ, координирует проверку остатков, инициирует транзакции оплаты и запускает отправку уведомлений.
- 3. Сервис инвентаризации (Inventory Service):** управляет складским учетом. Отвечает за проверку наличия товаров на складе, их временное резервирование и последующее списание остатков.
- 4. Платёжный сервис (Payment Service):** внешний интеграционный сервис, отвечающий за проведение транзакций через API банка-эквайера
- 5. Сервис уведомлений (Notification Service):** отвечает за формирование и отправку транзакционных писем (email-подтверждений) клиентам.
- 6. Брокер сообщений Apache Kafka:** шина данных, используемая для асинхронного, надежного обмена сообщениями о платежах.
- 7. Брокер сообщений RabbitMQ:** очередь сообщений, используемая для асинхронной отправки задач на email-уведомления.

Таблица 1 – Требования к взаимодействию компонентов

№	Компонент-ИС (точник)	Компонент-ПР (приёмник)	Тип взаимодействия	Протокол API	Формат данных	Доп. условия (таймаут, безопасность)
1	Сервис корзины	Сервис заказов	Синхронный	HTTP REST	JSON	Авторизация пользователя (JWT)
2	Сервис заказов	Сервис инвентаризации	Синхронный	HTTP POST / reserve	JSON	Таймаут: 2 секунды (безопасность склада)
3	Сервис заказов	Платежный сервис	Асинхронный	Apache Kafka (тема payments)	JSON / Avro	Гарантия доставки (at-least-once), шифрование TLS
4	Платежный сервис	Сервис заказов	Асинхронный	Apache Kafka (тема payment_results)	JSON / Avro	Обработка идемпотентности на стороне сервиса заказов
5	Сервис заказов	Сервис уведомлений	Асинхронный	AMQP (RabbitMQ)	JSON	Отказоустойчивая очередь (Durable), подтверждение доставки (Acks)

Диаграмма последовательности



Выводы

В ходе выполнения работы были проанализированы требования к взаимодействию микросервисов интернет-магазина.

1. **Критически важным аспектом** системы является синхронное резервирование товаров (Сервис заказов $\Rightarrow$ Сервис инвентаризации). Если этот шаг сделать асинхронным, возникнет риск «оверселлинга» (продажи товара, которого нет на складе).
2. **Асинхронность через Kafka и RabbitMQ** позволяет разгрузить систему. Платёжные шлюзы банков могут отвечать медленно, поэтому вынос оплаты и отправки писем в асинхронный режим гарантирует, что пользователь не будет долго ждать на экране «зависшую» страницу.

Контрольные вопросы:

1. Обязательные разделы ТЗ по ГОСТ 19.201-78

Согласно Единой системе программной документации (ЕСПД), в частности ГОСТ 19.201-78 «Техническое задание. Требования к содержанию и оформлению», обязательными структурными элементами являются:

- Вводная часть (аннотация).
- Основание для разработки.
- Назначение разработки.
- Требования к программе или программному изделию (*включая требования к функциональным характеристикам, надежности, условиям эксплуатации, составу и параметрам технических средств*).
- Требования к программной документации.
- Технико-экономические показатели.
- Стадии и этапы разработки.
- Порядок контроля и приемки.

Требования к взаимодействию компонентов обычно описываются в разделе «Требования к программе». Чаще всего это подраздел «Требования к интерфейсам» или «Требования к сопряжению с другими

системами/компонентами», где фиксируются протоколы, форматы данных и правила обмена информацией между модулями системы.

2. Функциональные и нефункциональные требования

Основное различие заключается в том, *что* система делает и *как* она это делает.

- **Функциональные требования** определяют поведение системы: конкретные функции, задачи, действия и реакции на ввод пользователя. Они отвечают на вопрос «Что система должна делать?». *Пример:* Система должна сохранять данные о заказе клиента в базу данных при нажатии кнопки «Оформить заказ».
- **Нефункциональные требования** определяют атрибуты качества, ограничения и условия работы системы. Они не описывают конкретную функцию, а задают рамки её выполнения. Отвечают на вопросы «Как быстро?», «Насколько надежно?», «В каком формате?» и т.д.

Пример нефункционального требования к взаимодействию:

«Время отклика API для получения списка товаров не должно превышать 200 миллисекунд при нагрузке до 500 запросов в секунду». Это требование описывает производительность взаимодействия, но не саму логику запроса.

3. Виды взаимодействия компонентов: синхронный и асинхронный обмен

Существует два основных подхода к организации связи между компонентами:

- **Синхронное взаимодействие:** Вызывающий компонент отправляет запрос и блокирует свою работу, ожидая немедленного ответа от вызываемого компонента. Запрос и ответ происходят последовательно в рамках одной операции.
 - *Когда используется:* Когда результат операции необходим немедленно для продолжения работы (например, авторизация пользователя перед входом в систему, получение баланса счета). Если ответ не придет вовремя, работа приложения остановится.
- **Асинхронное взаимодействие:** Вызывающий компонент отправляет сообщение (событие) и продолжает свою работу, не дожидаясь ответа. Ответ может быть получен позже через callback-функцию, очередь сообщений или отдельное событие.

- *Когда используется:* Для задач, которые могут выполняться долго или не требуют мгновенного результата (отправка email-рассылок, генерация отчетов, обработка видео, обновление поискового индекса). Это повышает отказоустойчивость и масштабируемость системы.

4. Что такое API и как его описать в тексте ТЗ

API (Application Programming Interface) — это набор правил, протоколов и инструментов, который позволяет разным программным компонентам или внешним сервисам взаимодействовать друг с другом.

Описание API в тексте технического задания включает следующие элементы:

- **Назначение API:** Краткое описание того, какие возможности предоставляет интерфейс.
- **Методы (HTTP-методы):** GET, POST, PUT, DELETE и др.
- **Эндпоинты (URL-пути):** Например, /api/v1/users.
- **Запросы:** Формат тела запроса (обычно JSON), структура параметров (в URL или заголовках), примеры.
- **Ответы:** Структура возвращаемых данных (JSON/XML), коды состояния HTTP (200 OK, 404 Not Found, 500 Internal Server Error), примеры успешных ответов и ошибок.
- **Авторизация:** Способ проверки прав доступа (например, JWT-токены, OAuth).

5. Важность указания протокола и формата данных

Указание этих параметров критически важно для корректной интеграции, так как они обеспечивают единый "язык общения" между компонентами.

- **Протокол** (например, HTTP, gRPC, AMQP) определяет правила установления соединения, передачи данных и обработки ошибок. Без единого протокола компоненты просто не смогут "услышать" друг друга.
- **Формат данных** (например, JSON, XML, Protocol Buffers) определяет синтаксис сообщения. Если один модуль отправит данные в формате JSON, а другой ожидает XML, произойдет ошибка парсинга, и информация будет потеряна.

Четкое определение этих аспектов исключает двусмысленность и гарантирует, что разработчики разных частей системы создадут совместимые модули.

6. Проблемы при нечёткой формулировке требований к взаимодействию

Если в ТЗ указано лишь «модули должны обмениваться данными» без деталей, возникают следующие проблемы:

- **Несовместимость форматов:** Один модуль формирует данные в виде CSV-файла, а другой ждет поток бинарных данных.
- **Проблемы производительности:** Разработчик реализует простой, но медленный способ передачи (например, запись в общую файловую папку), который не справляется с высокой нагрузкой, хотя требовалась быстрая шина данных.
- **Ошибки синхронизации:** Неясно, должен ли вызывающий модуль ждать завершения операции (синхронно) или продолжать работу (асинхронно), что приводит к непредсказуемому поведению программы.
- **Сложности с безопасностью:** Отсутствие требований к шифрованию и аутентификации приведет к передаче чувствительных данных в открытом виде.
- **Увеличение стоимости и сроков:** Программистам придется тратить время на бесконечные согласования, переделку уже написанного кода и исправление багов интеграции, возникающих из-за неверных предположений.